

## 08: Time Series – Machine Learning Approach

Forecasting as a supervised learning problem

## Where we left off

Last time: ARIMA, SARIMA, exponential smoothing – purpose-built statistical models. They are excellent, and often hard to beat on a single clean series.

**Today's idea:** we already own a toolbox of powerful regressors – linear models, random forests, gradient boosting, neural nets. What if we could point *those* at a time series?

We can. The trick is to turn “predict the next value” into an ordinary  $(\mathbf{X}, \mathbf{y})$  table. Once it's a table, every model from this course applies.

The catch: the i.i.d. assumption is still false, so we have to be very careful about *how* we build the table and *how* we validate. That discipline is most of this lecture.

# Outline

Forecasting as supervised learning

Leakage and time-aware splitting

Feature engineering for time

Time-aware cross-validation

Tree models and the extrapolation trap

So which wins?

Deep learning and foundation models

Recap

## Forecasting as supervised learning

Slide a window over the series and out falls a design matrix.

## The reframing

Pick a target “now” ( $y_t$ ) and describe it with values from its own past ( $y_{t-1}, y_{t-2}, \dots$ ). Each time step becomes one training row.

Forecasting IS supervised learning: sliding a window makes (X, y) rows

| y(t-3) | y(t-2) | y(t-1) | y(t) = target |
|--------|--------|--------|---------------|
| 108    | 114    | 121    | 118           |
| 114    | 121    | 118    | 120           |
| 121    | 118    | 120    | 116           |
| 118    | 120    | 116    | 108           |
| 120    | 116    | 108    | 99            |
| 116    | 108    | 99     | 86            |

That's it – the columns on the left are features, the right column is the label. Feed it to `LinearRegression`, `RandomForest`, `HistGradientBoosting` – whatever you like.

## Leakage and time-aware splitting

The single most common way to fool yourself with time series.

## Predict-first: what's wrong with a random split?

You have your  $(\mathbf{X}, \mathbf{y})$  table. Instinct says `train_test_split(X, y, shuffle=True)`.

**Predict: what does a random split do to a time series?**

## Predict-first: what's wrong with a random split?

You have your  $(\mathbf{X}, \mathbf{y})$  table. Instinct says `train_test_split(X, y, shuffle=True)`.

**Predict: what does a random split do to a time series?**

It puts *future* rows in the training set and *past* rows in the test set. The model peeks at the future to “predict” the past. Your test score is a fantasy.

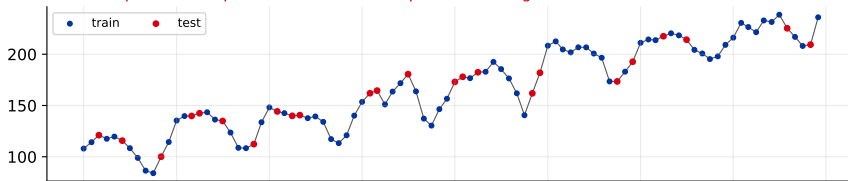
Worse, the rows aren't even independent:  $y_t$  and  $y_{t-1}$  share almost everything, so a random test point sits right next to its own neighbours in train. Textbook **data leakage**.

The fix is boring and non-negotiable: **split by time**.

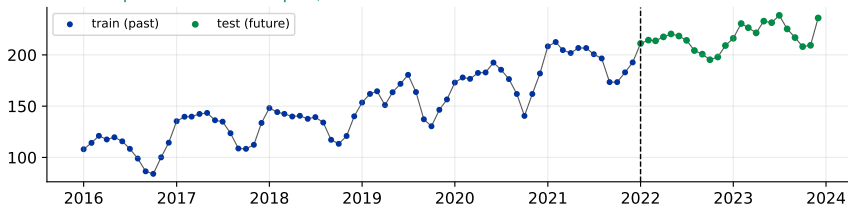


# Random split vs forward split

Random split -> test points sit BEFORE train points = leakage



Forward split -> train on the past, test on the future = honest



Train on an earlier block, test on the later block. Same rule as lecture 07: only ever learn from the past. This also means **no shuffling**, and any scaler / encoder must be fit on train only.

## Feature engineering for time

This is where an ML forecaster is won or lost.

# The four families of time features

## 1. Lags – past values.

- ▶ `y.shift(1), shift(7), shift(12)`.
- ▶ Pick lags at meaningful horizons (yesterday, last week, last year).

## 2. Rolling / window stats – local summaries.

- ▶ `y.shift(1).rolling(7).mean() / std / min / max`.
- ▶ **Always shift before rolling** – else the window includes  $y_t$  itself = leakage.

Golden rule: every feature for row  $t$  must be computable using *only* information available strictly before  $t$ .

## 3. Calendar / date parts – known in advance.

- ▶ month, day-of-week, hour, is\_weekend, is\_holiday.
- ▶ Cyclically encode:  $\sin / \cos(2\pi \cdot \text{hour}/24)$ .

## 4. Exogenous regressors – outside drivers.

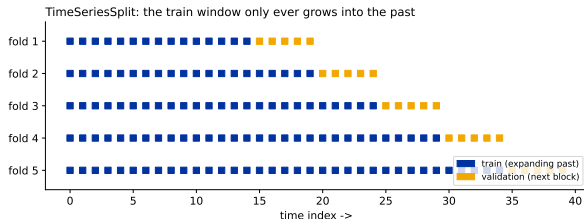
- ▶ price, promotion, weather, events.
- ▶ The big advantage of ML over plain ARIMA.

## **Time-aware cross-validation**

K-fold, but respecting the arrow of time.

# TimeSeriesSplit

Ordinary K-fold shuffles – forbidden here. Use forward-chaining: each fold trains on an expanding past and validates on the next block.



```
from sklearn.model_selection \
    import TimeSeriesSplit
cv = TimeSeriesSplit(n_splits=5)
cross_val_score(model, X, y, cv=
cv)
```

This is also how you tune hyperparameters and pick lag sets – grid/random search with `cv=TimeSeriesSplit`, never plain K-fold.

## **Tree models and the extrapolation trap**

Gradient boosting is the tabular champion – with one sharp edge.

## Predict-first: point GBM at a trending series

Gradient boosting (XGBoost / LightGBM / HistGradientBoosting) wins most tabular competitions. We built lag + calendar features. Let it forecast the trending series from lecture 07.

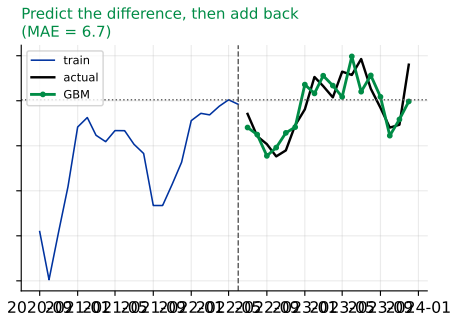
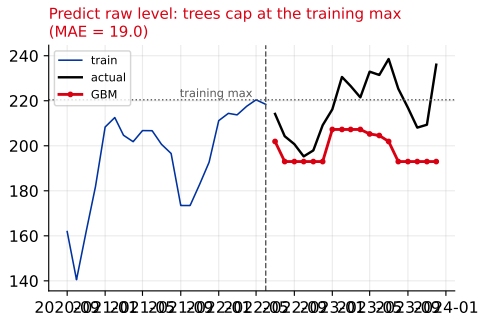
**Predict: does the boosting model track the upward trend?**

## Predict-first: point GBM at a trending series

Gradient boosting (XGBoost / LightGBM / HistGradientBoosting) wins most tabular competitions. We built lag + calendar features. Let it forecast the trending series from lecture 07.

**Predict: does the boosting model track the upward trend?**

Gradient boosting on a trending series: the extrapolation trap and its fix





## Trees cannot extrapolate – so difference first

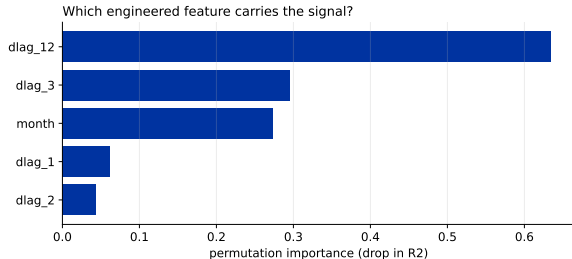
A decision tree predicts by averaging *training* labels in a leaf. Its output can never exceed the largest  $y$  it saw in training.

On a trending series the future is always above that ceiling, so a raw-level tree model **flat-lines** below the trend (left panel, MAE 19).

**The fix:** make the target stationary first – predict the *difference*  $y_t - y_{t-1}$  (or a ratio), then add it back. Now the targets live in a fixed range trees can handle (right panel, MAE 6.7).

Same lesson as ARIMA's "I": difference away the trend. Linear models and neural nets extrapolate fine; *trees do not*.

# Which features actually mattered?



Permutation importance on the differenced model:

- ▶ `dlag_12` – the seasonal lag – dominates.
- ▶ `dlag_3` and `month` carry the rest.

This is why domain knowledge beats brute force: knowing the season is 12 tells you which lag to build. The model can only use features you gave it.

## Forecasting more than one step ahead

For horizon  $h > 1$  you have two strategies:

**Recursive** – predict  $t+1$ , feed it back in as a lag, predict  $t+2$ , ...

- ▶ One model, reused.
- ▶ Errors compound down the horizon.

**Direct** – train a separate model per horizon ( $h=1$  model,  $h=2$  model, ...).

- ▶ No error feedback.
- ▶ More models to train and store.

You rarely hand-code this. Libraries like `skforecast`, `mlforecast` (Nixtla), `sktime` and `Darts` wrap any scikit-learn regressor into a recursive/direct forecaster with the leakage-safe plumbing built in.

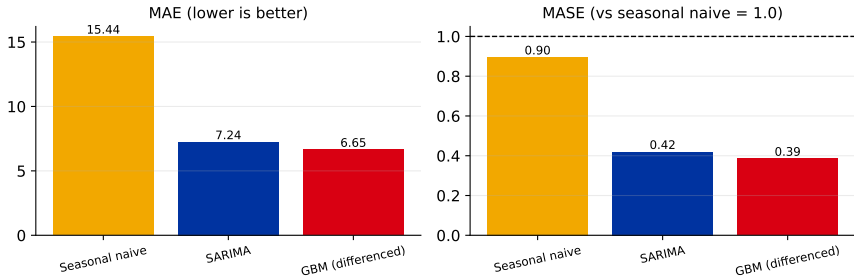
## So which approach wins?

Classical vs ML – the honest answer.

## Benchmark everything against the naive baseline

Our three contenders on the held-out year – seasonal naive, SARIMA, and the (differenced) gradient booster:

Always benchmark against the naive baseline



On this short, clean series SARIMA and GBM are neck-and-neck, both crushing seasonal naive (MASE  $\approx 0.4$  vs 0.9). The point is the *habit*: report MASE against the naive baseline, always.

# When to reach for ML vs classical

## **Classical (ARIMA / ETS) shines**

- ▶ one or a few series,
- ▶ short history,
- ▶ strong, stable seasonality,
- ▶ you need calibrated prediction intervals cheaply.

## **ML (boosting / NN) shines**

- ▶ many related series (retail: 10k products),
- ▶ long history,
- ▶ rich exogenous features (price, weather, promos),
- ▶ non-linear interactions between drivers.

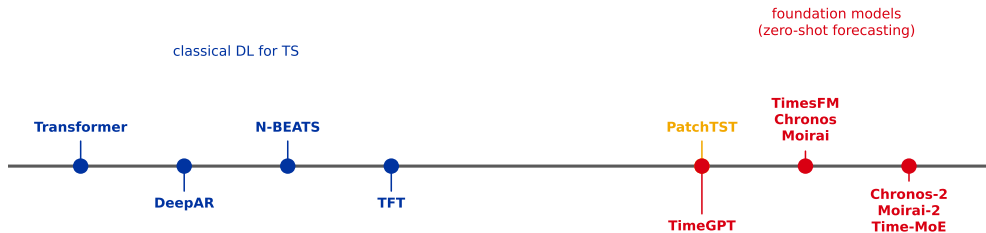
In the M5 forecasting competition (Walmart sales, thousands of series), gradient-boosted trees on engineered features won. On the earlier M3/M4 single-series benchmarks, statistical methods and simple combinations were extremely strong. Match the tool to the shape of the problem.

## Deep learning and foundation models

Awareness level – what exists and when it's worth it.

# The deep learning lineage

## Deep and foundation models for time series



### Specialist deep models

- ▶ **DeepAR** – probabilistic RNN over many series.
- ▶ **N-BEATS / N-HiTS** – pure MLP stacks.
- ▶ **TFT** – attention + interpretability.
- ▶ **PatchTST** – patch the series, feed a Transformer.

### Why they connect to us

These pick up in the RNN and attention chapters. They need a lot of data (many series, long history) to beat a well-tuned gradient booster – on a single short series they usually don't.



# Foundation models: zero-shot forecasting

The 2023–2025 shift: pre-train one Transformer on billions of time points, then forecast a *new* series with no training at all – like an LLM prompt.

## The current crop

- ▶ **TimesFM** (Google) – patched decoder, pre-trained on 100B points.
- ▶ **Chronos / Chronos-2** (Amazon) – tokenizes values like language; Chronos-Bolt is much faster.
- ▶ **Moirai / Moirai-2** (Salesforce) – multivariate, any frequency.
- ▶ **TimeGPT** (Nixtla), **Lag-Llama**, **Time-MoE**.

**Reality check.** Zero-shot is astonishing for a cold start with no history. But a gradient booster with good features and exogenous drivers is still a very strong, cheap, interpretable baseline. Try the simple thing first.

## Recap – ML for time series

- ▶ Forecasting → supervised learning by sliding a window.
- ▶ **Never** random-split; split and cross-validate by time (`TimeSeriesSplit`).
- ▶ Features: lags, (shifted) rolling stats, calendar, exogenous.
- ▶ Trees can't extrapolate – difference the target first.
- ▶ Multi-step: recursive vs direct (let a library handle it).
- ▶ Report MASE vs the naive baseline.
- ▶ Deep + foundation models: powerful, data-hungry, know they exist.

### The one habit to keep:

every feature and every split must respect the arrow of time. If a value wasn't knowable yet, it can't be in the row.

**Toolbox:** `sktime`, `statsforecast` & `mlforecast` (Nixtla), `skforecast`, Darts, Prophet, GluonTS.

## Questions?

Homework: build a forecaster the leakage-safe way. See the chapter page.